

Práctica. CWE-476: NULL Pointer Dereference

Instrucciones. Elaborar una aplicación con Node.js que permita ejemplificar la vulnerabilidad Desreferencia de puntero NULL en una aplicación web cliente-servidor.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de practica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-476.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente en Visual Studio Code del archivo `app.js`.
 - b. Coloca la captura de pantalla de la salida de la página `index.html` que lee el id de un alumno.
 - c. Coloca la captura de pantalla de la salida del error de Node.js al intentar acceder a un objeto no definido.
 - d. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para EJS.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
2. Esta práctica tiene como prerrequisito tener instalado la plataforma Node.js en su equipo.
Puede seguir las instrucciones de cómo instalar Node.js desde la práctica [Instalación de Node.js](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

Instrucciones

Una desreferencia de puntero NULL ocurre cuando la aplicación desreferencia un puntero que espera que sea válido, pero es NULL, lo que generalmente provoca un bloqueo o una salida. En este ejemplo, al no realizar una validación de entrada apropiada, produce que un atacante pueda producir una denegación del servicio de una aplicación web.

Para este ejemplo, utilizaremos la plataforma Node.js junto con el paquete **Express**, la cual es un web application framework para Node.js mínimo y flexible que proporciona un conjunto sólido de funciones para desarrollar aplicaciones web y móviles. Facilita el rápido desarrollo de aplicaciones web basadas en Nodos. Las siguientes son algunas de las características principales del marco Express:

- Permite configurar middlewares para responder a solicitudes HTTP.
- Define una tabla de enrutamiento que se utiliza para realizar diferentes acciones según el método HTTP y la URL.
- Permite representar dinámicamente páginas HTML basándose en pasar argumentos a plantillas.

A. Crear el programa servidor

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
2. Seleccione **Archivo > Abrir carpeta** en el menú principal.
3. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-476** y selecciónela. Luego haga clic en Seleccionar carpeta.
4. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
5. Abra una Terminal de consola presionando Ctrl+n.
6. Utilice el comando **npm init** para crear un archivo **package.json** para su aplicación.

```
> npm init
```

7. Este comando le solicita varias cosas, como el nombre y la versión de su aplicación. Por ahora, puedes simplemente presionar **Enter** para aceptar los valores predeterminados para la mayoría de ellos, con la siguiente excepción:

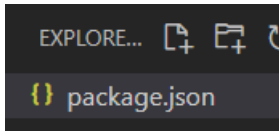
```
entry point: (index.js)
```

8. Ingrese **app.js**, que será el nombre del archivo principal. En **Node.js** se recomienda que ese sea el nombre del archivo principal.

```
package name: (server)
version: (1.0.0)
description: cwe-476
entry point: (index.js) app.js
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to C:\codigo\ps\cwe-476\server\package.json:
{
  "name": "server",
```

```
"version": "1.0.0",
"description": "cwe-476",
"main": "app.js",
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"author": "",
"license": "ISC"
}
Is this OK? (yes)
```

9. Observe que se ha creado un archivo llamado `package.json`. Si quisiera omitir el asistente, solo se agrega la flag `-y` al comando `npm init -y`. Deje abierto este archivo.



10. Ahora instale el paquete **Express** en este directorio y guárdelo en la lista de dependencias del proyecto. Para instalarlo temporalmente y no agregarlo a la lista de dependencias agregue `--no-save`.

```
> npm install express --save
```

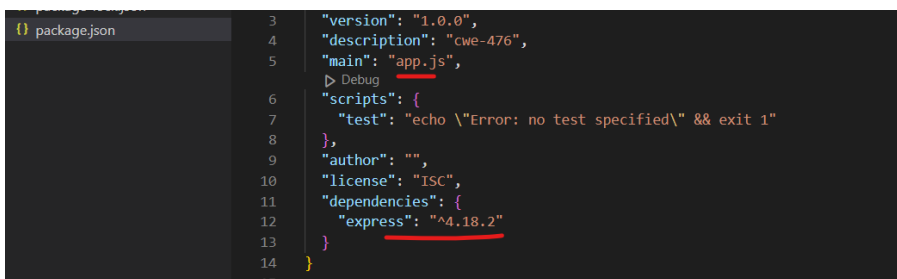
```
PS C:\codigo\ps\cwe-476\server> npm install express

added 62 packages, and audited 63 packages in 3s

11 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\codigo\ps\cwe-476\server> 
```

11. Observe que se ha instalado la versión `4.18.2` de Express.



12. Solo como referencia, para desinstalar un paquete, se utiliza `npm uninstall <package_name>`.
13. Cree una nueva carpeta llamada `public`. Ahí es donde colocaremos los archivos estáticos como HTML, imágenes, scripts u hojas de estilo.
14. Dentro de esta carpeta, agregue un archivo llamado `index.html` y escriba el siguiente código.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link rel="icon" href="data:;base64,iVBORw0KGgo=" />
<title>Programación segura - Universidad Veracruzana</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="container mt-4">
  <h2>Programación segura <small class="text-muted fs-5">Universidad Veracruzana</small></h2>

  <form action="/alumno" method="post">
    <div class="card mx-auto mt-5 col-lg-6">
      <div class="card-header">Selección de alumnos</div>
      <div class="card-body">
        <div class="mb-3">
          <label for="id" class="form-label">Seleccione un id de alumno:</label>
          <select name="id" id="id">
            <option value="1">1</option>
            <option value="2">2</option>
            <option value="3">3</option>
          </select>
        </div>
      </div>
      <div class="card-footer">
        <button id="submit" type="submit" class="btn btn-primary">Obtener nombre</button>
      </div>
    </div>
  </form>
</body>

</html>
```

15. Este archivo es la interfaz principal de nuestro ejemplo. Sería nuestro frontend.
16. Ahora vamos a crear el backend en Node.js. Cree un nuevo archivo llamado `app.js` y agregue el siguiente código.

```
const express = require('express')
const app = express()
const port = 3000

app.use(express.static("public"))
app.use(express.urlencoded({ extended: true }))

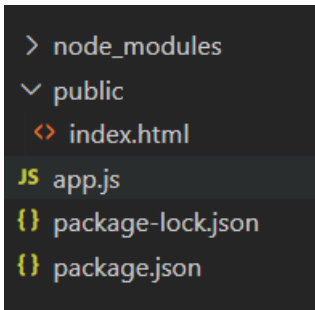
app.post('/alumno', (req, res) => {
  if(req.body.id)
    res.send(`Hola ${alumnos[req.body.id - 1].nombre} ${alumnos[req.body.id - 1].apellido}`);
  else
    res.send('No se ha especificado el id del alumno.');
```

```
  })

app.listen(port, () => {
  console.log(`Aplicación de ejemplo escuchando en el puerto ${port}`)
})

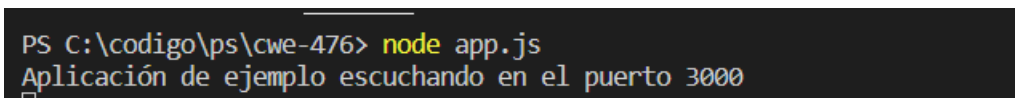
const alumnos = [
  { nombre: "Pedro", apellido: "Perez" },
  { nombre: "Rosa", apellido: "Vazquez" },
  { nombre: "Susana", apellido: "Ochia" }
];
```

17. Esta aplicación inicia un servidor web y escucha las conexiones en el puerto 3000.
18. Su proyecto debería tener los siguientes archivos.

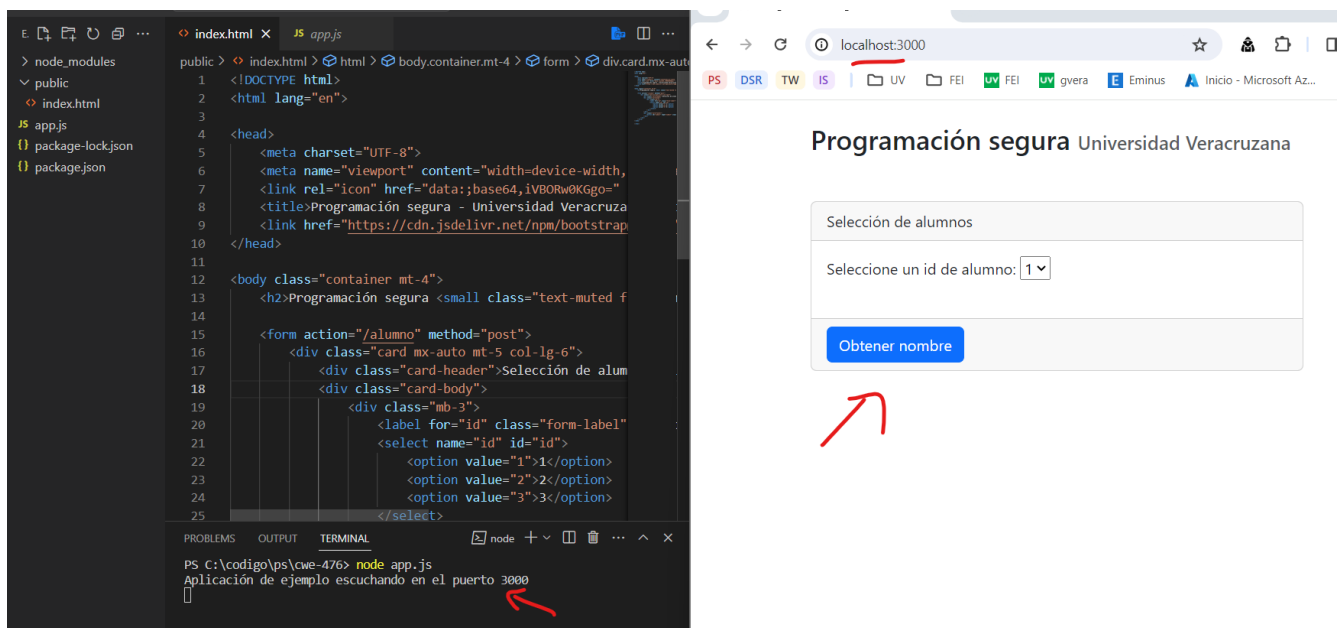


19. Ejecute la aplicación con el siguiente comando desde la terminal de VS Code.

```
> node app.js
```



20. Abra un navegador web y abra la ruta <http://localhost:3000/> para ver el resultado.



21. Seleccione algún valor del cuadro de lista y presione el botón **Obtener nombre**.

Hola Pedro Perez

22. Observe como se imprime el nombre del usuario con el **id** seleccionado. La aplicación funciona correctamente.
23. Si cambia el id, cambia el usuario. Solo se están publicando los id de los usuarios que existen en el sistema, por lo que no hay lugar para la selección de algún usuario incorrecto.
24. Enhorabuena. Ha creado su aplicación de manera correcta.

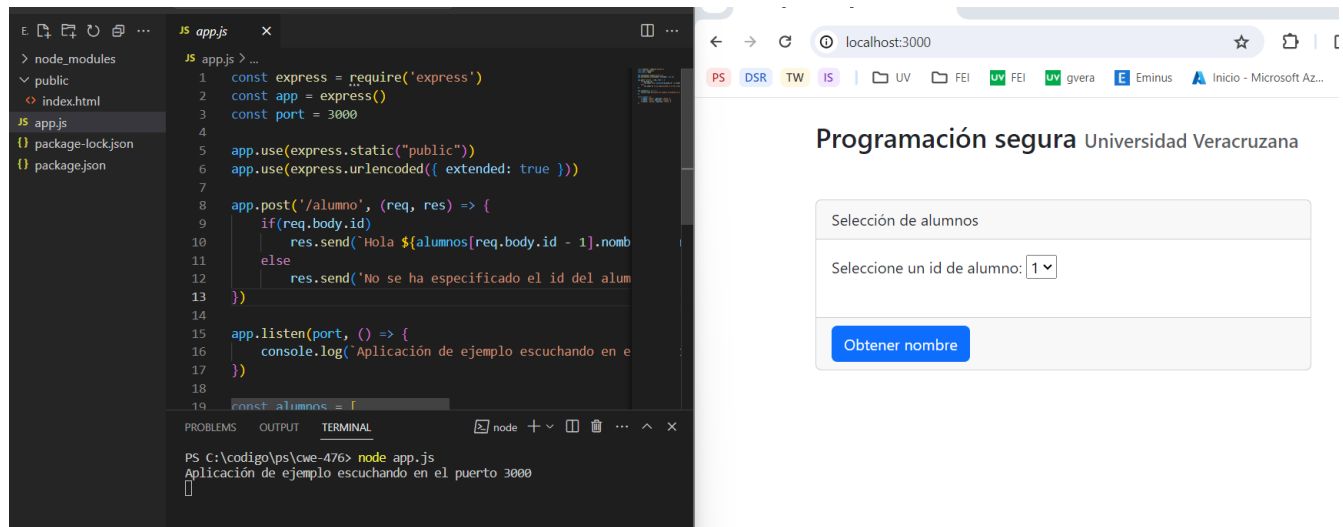
B. Explotar la vulnerabilidad

En este ejemplo vamos a encontrar el punto de entrada de la vulnerabilidad en la validación de valores nulos. Es muy común que la validación se haga pensando que el usuario no puede alterar los datos enviados. Por ejemplo, en este código, se realiza una validación de valores nulos sobre el parámetro enviado por POST.

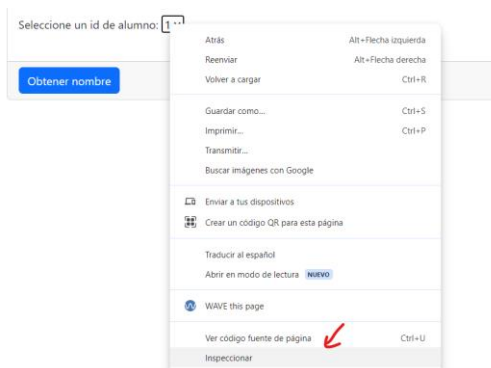
```
app.post('/alumno', (req, res) => {  
  if(req.body.id)  
    res.send(`Hola ${alumnos[req.body.id - 1].nombre} ${alumnos[req.body.id - 1].apellido}`);  
  else  
    res.send('No se ha especificado el id del alumno.');
```

Primeramente vamos a intentar enviar un valor **nulo** para el valor de **id**. Si no funciona, vamos a intentar enviar otro valor para que el arreglo de alumnos haga referencia a un valor de memoria **nulo**.

1. Abra VS Code ejecutando su servidor de Node.js del lado izquierdo de su pantalla. Abra del lado derecho su aplicación web corriendo en <http://localhost:3000/>.



2. Haga clic derecho sobre el control de lista y seleccione **Inspeccionar**.



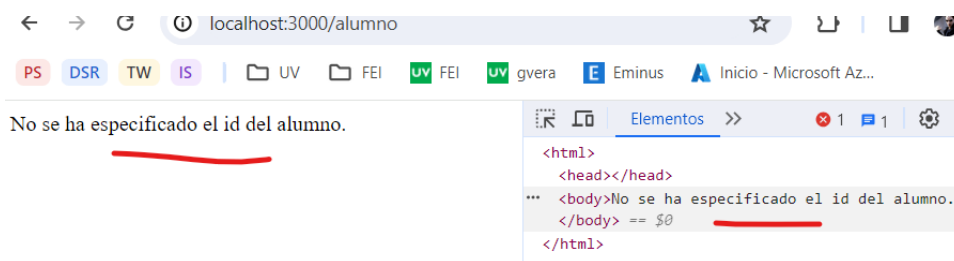
3. Observe como se muestra el código HTML que dibuja el control de lista.



4. Vamos a modificar el valor de la opción 2 y vamos a ponerle vacío. Para enviar un id nulo al servidor.
5. Haga doble clic para entrar en modo de edición y borre el valor de "2" del value y presione Enter.

```
<option value="1">1</option>
<option value="2">2</option>
<option value="3">3</option>
```

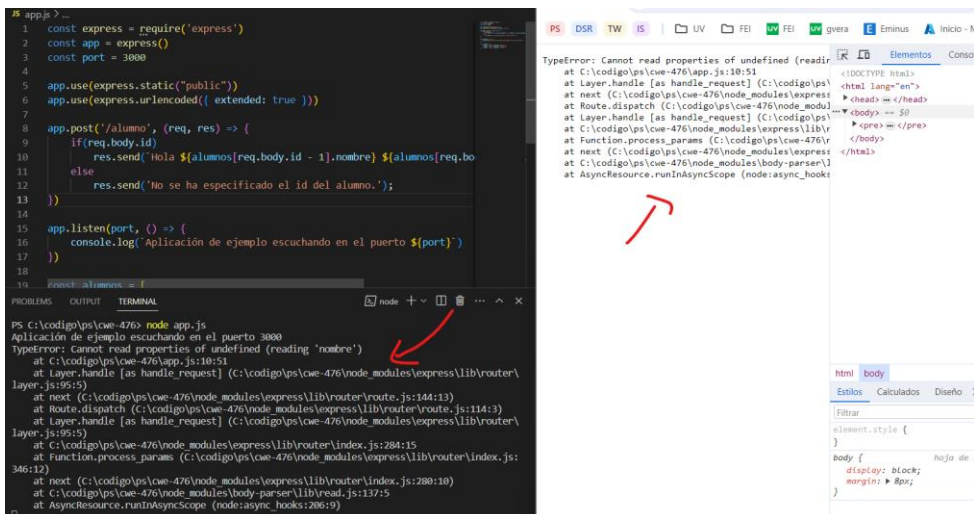
6. Ahora en la página seleccione el elemento 2 del cuadro de lista y presione el botón **Obtener nombre**.



7. Esto quiere decir que la validación del lado del servidor funciona.
8. Presione el botón regresar de su navegador. Vuelva editar el control de lista, ahora la opción 3. Coloque el valor de -2.

```
<option value="2">2</option>
<option value="-2">3</option>
```

9. Presione el botón de **Enter**.
10. Observe como hemos conseguido explotar la vulnerabilidad y detener la ejecución del sitio.



11. Hemos provocado la caída del sitio con un error de referenciación a un objeto nulo del arreglo de alumnos.

```
Aplicacion de ejemplo escuchando en el puerto 3000
TypeError: Cannot read properties of undefined (reading 'nombre')
    at C:\codigo\ps\cwe-476\app.js:10:51
    at Layer.handle [as handle_request] (C:\codigo\ps\cwe-476\node_modules\express\lib\router\layer.js:95:5)
    at next (C:\codigo\ps\cwe-476\node_modules\express\lib\router\route.js:144:13)
    at Route.dispatch (C:\codigo\ps\cwe-476\node_modules\express\lib\router\route.js:114:3)
    at Layer.handle [as handle_request] (C:\codigo\ps\cwe-476\node_modules\express\lib\router\layer.js:95:5)
    at C:\codigo\ps\cwe-476\node_modules\express\lib\router\index.js:284:15
    at Function.process_params (C:\codigo\ps\cwe-476\node_modules\express\lib\router\index.js:346:12)
    at next (C:\codigo\ps\cwe-476\node_modules\express\lib\router\index.js:280:10)
    at C:\codigo\ps\cwe-476\node_modules\body-parser\lib\read.js:137:5
    at AsyncResource.runInAsyncScope (node:async_hooks:206:9)
```

12. Felicidades, ha creado su aplicación de manera correcta.